Sensitivity to Neural Network Tuning

BZAN 554 Group Assignment 2 · Group 6

Joseph Falcone · Ian Allish · Will Schneider · Noah Hoogerheide

Study objective and approach

How does neural network tuning affect prediction accuracy?

1 Start at baseline

Use the same reference model in each experiment.

→

2 Change one setting

Compare each tested setting against the baseline.

→

3 Reset the model

Start the next experiment from baseline settings.

Data and preprocessing

One row represents one product selling episode

SKU, price, order, duration, and category predict quantity.



Random row split · Seed 42

Fit min-max scaling and category and SKU mappings on training rows only. Scale numeric inputs and quantity, one-hot encode category, and embed SKU in eight dimensions. Load 16,384 rows per chunk for incremental training.

Repeated SKUs can occur across splits. The comparison concerns held-out episodes, including episodes for products seen in training.

Training and evaluation

1 Train

20 epochs, seed 42, and a fixed learning rate within each run.

→

2 Update

Shuffle training rows within each chunk and update by mini-batch.

→

3 Select

Check validation MSE each epoch and restore the best weights. All runs complete 20 epochs.

75,053 validation observations

RMSE and MAE use the supplied quantity scale.

Source: We tested 31 configurations on both October 30 and 31. Scores match across dates. Charts use the October 31 runs, including runtime.

These repeated runs use the same seed and do not measure variability across seeds. The CSV contains validation results only.

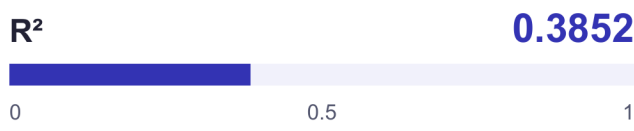
Baseline model

Three sigmoid hidden layers: **21 / 12 / 7 neurons**. Eight-dimensional SKU embedding, one-hot category, scaled numeric inputs, and a linear output.

Baseline settings

Optimizer	**Adam**
Learning rate	**0.001**
Batch size	**1,024**
Selected epoch	**19**

Validation performance

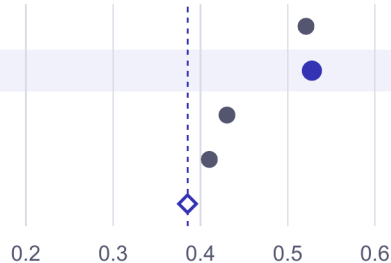


MSE	RMSE	MAE
1,612.48	**40.16**	**21.40**

All following ΔR^2 values compare with this baseline. Positive values indicate improvement.

Batch size

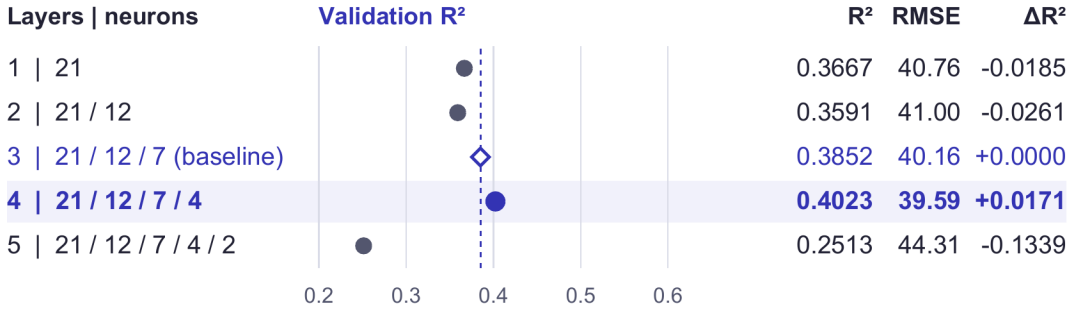
Batch size	R^2	RMSE	ΔR^2	Sec.
32	0.5210	35.44	+0.1358	187.4
128	**0.5277**	**35.20**	**+0.1425**	**61.2**
256	0.4304	38.65	+0.0452	39.6
512	0.4102	39.33	+0.0250	28.5
1,024 (baseline)	0.3852	40.16	+0.0000	22.9



Finding: Batch 128 has the highest validation R^2 in this comparison, but takes about 2.7 times the baseline runtime.

Interpretation: Smaller batches make more weight updates per epoch. This comparison includes that extra optimization work. Runtime covers preprocessing and the training loop, including validation.

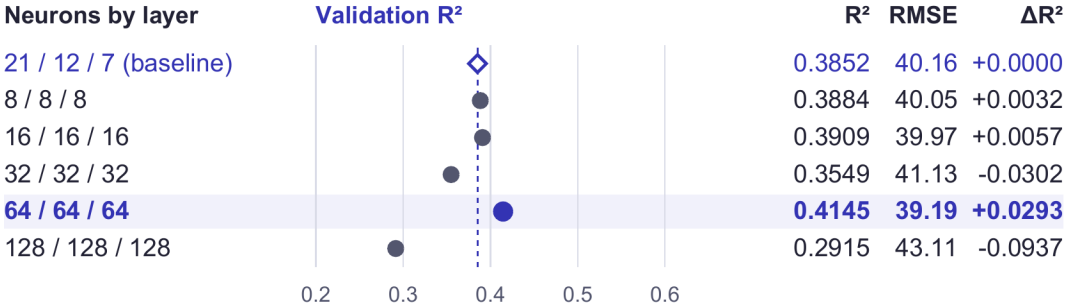
Number of hidden layers



Finding: Four layers improve R^2 slightly. Five layers perform substantially worse.

Interpretation: Extra depth does not guarantee better predictions. The narrow added layers also change capacity, so this result applies to these specific architectures.

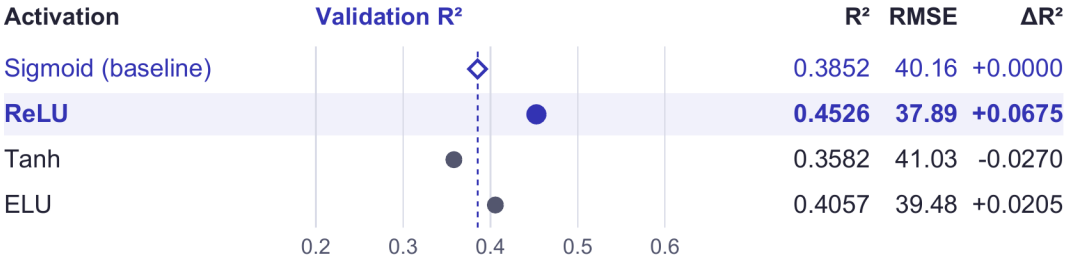
Neurons per hidden layer



Finding: The 64 / 64 / 64 network performs best among the tested widths. The widest network performs worse than the baseline.

Interpretation: More neurons change capacity and optimization. Scores do not improve steadily with width. These trials also replace the baseline's taper with equal-width layers.

Hidden activation function



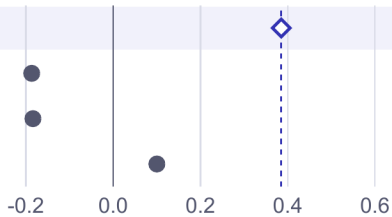
Finding: ReLU gives the largest gain in this comparison. Tanh reduces R^2.

Interpretation: ReLU avoids sigmoid's positive-side saturation, which can help gradient flow. This is a possible explanation for the gain, not a measured gradient result.

The output layer remains linear for quantity regression.

Optimizer

Optimizer	R²	RMSE	ΔR²
Adam (baseline)	**0.3852**	**40.16**	**+0.0000**
SGD	-0.1866	55.79	-0.5718
AdaGrad	-0.1838	55.72	-0.5690
RMSprop	0.1002	48.58	-0.2850

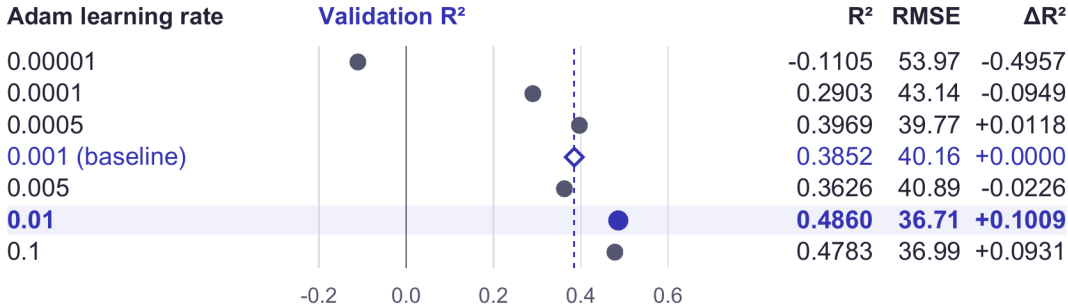


Finding: Adam performs best with the shared learning rate of **0.001**. SGD and AdaGrad have negative validation R^2.

Negative R^2 means greater squared error than predicting the validation-set mean.

Interpretation: Optimizers behave differently at the same learning rate. These runs compare a shared setting rather than each optimizer's best possible tuning.

Learning rate

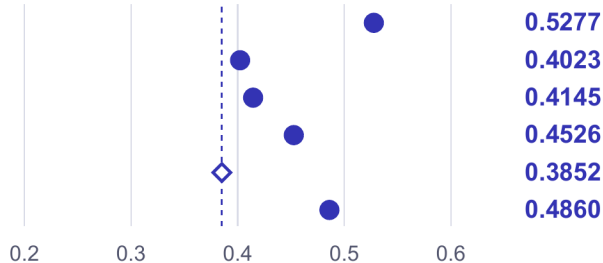


Finding: 0.01 performs best among these rates. The smallest rate produces negative R^2.

Interpretation: A very small rate may learn too slowly within the shared training budget. Larger rates help here, but the relationship is not monotonic.

Sensitivity findings

Comparison	Best setting tested	R^2
Batch size	128	**0.5277**
Hidden layers	4 | 21 / 12 / 7 / 4	**0.4023**
Hidden neurons	64 / 64 / 64	**0.4145**
Activation	ReLU	**0.4526**
Optimizer	**Adam (baseline)**	**0.3852**
Learning rate	0.01	**0.4860**



Learning rate and optimizer produce the largest observed spreads in R^2 across their tested settings.

Each row summarizes a separate comparison against baseline R^2 **0.3852**. These settings were not combined or confirmed as a final model.

Scope and limitations

Repeatability	One random seed and one split limit conclusions about repeatability.
Training budget	Equal epochs give smaller batches more updates. Runtime also varies.
Validation only	Validation selects the best epoch and supports these comparisons. The CSV contains validation scores only.
Interactions	Each run changes one setting. Interactions, such as ReLU with a different learning rate, are unmeasured.
Untested settings	Learning-rate schedules, dropout, L2, gradient clipping, and sequence models are outside these runs.

Conclusion: Performance is sensitive to tuning choices. More layers, more neurons, or a smaller learning rate do not consistently improve validation results.

Team contributions

Team member	Team Contributions
Ian Allish	**1–4:** Objective and approach, data, training
Joseph Falcone	**5–8:** Baseline, batch size, layer count, neuron count
Will Schneider	**9–11:** Activations, optimizers, learning rates
Noah Hoogerheide	**12–14:** Findings, limitations, team roles

AI assistance: Codex helped revise this presentation.